

P2079R5

System execution context

Published Proposal, 2024-10-16

This version:

<http://wg21.link/P2079R5>

Authors:

[Lucian Radu Teodorescu](#)

[Ruslan Arutyunyan](#) (Intel)

[Lee Howes](#)

[Michael Voss](#) (Intel)

Audience:

SG1, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

A standard execution context based on the facilities in [\[P2300R10\]](#) that implements parallel-forward-progress to maximize portability. A set of `system_context`'s share an underlying shared thread pool implementation, and may provide an interface to an OS-provided system thread pool.

Table of Contents

1	Changes
1.1	R5
1.2	R4
1.3	R3
1.4	R2
1.5	R1

1.6 R0

2 Introduction

2.1 Design overview

3 Examples

4 Design

5 Design discussion and decisions

5.1 To drive or not to drive

5.2 Freestanding implementations

5.3 Making system context implementation-defined and replaceable

5.4 Extensibility

5.5 API for implementing custom system contexts

5.6 Shareability

5.7 Performance

5.8 Lifetime

5.9 Need for the `system_context` class

5.10 Priorities

5.11 Reference implementation

5.12 Addressing received feedback

5.12.1 Allow for system context to borrow threads

5.12.2 Allow implementations to use Grand Central Dispatch and Windows Thread Pool

5.12.3 Priorities and elastic pools

5.12.4 Implementation-defined may make things less portable

6 Annex: A possible API for implementing custom system contexts

References

Informative References

§ 1. Changes

§ 1.1. R5

- Streamline the paper
- Make replaceability availability implementation-defined
- Make replaceability API implementation-defined
- Replace the use of `system_context` class with direct calls to `get_system_scheduler()`
- Update user-facing API
- Relax the lifetime guarantees to allow using system scheduler outside of `main()`

§ 1.2. R4

- Add more design considerations & goals.
- Add comparison of different replaceability options
- Add motivation for replaceability ABI standardization
- Add the example of the ABI for replacement
- Strengthen the lifetime guarantees.

§ 1.3. R3

- Remove `execute_all` and `execute_chunk`. Replace with compile-time customization and a design discussion.
- Add design discussion about the approach we should take for customization and the extent to which the context should be implementation-defined.
- Add design discussion for an explicit `system_context` class.
- Add design discussion about priorities.

§ 1.4. R2

- Significant redesign to fit in [\[P2300R10\]](#) model.
- Strictly limit to parallel progress without control over the level of parallelism.

- Remove direct support for task groups, delegating that to `async_scope`.

§ 1.5. R1

- Minor modifications

§ 1.6. R0

- First revision

§ 2. Introduction

[P2300R10] describes a rounded set of primitives for asynchronous and parallel execution that give a firm grounding for the future. However, the paper lacks a standard execution context and scheduler. It has been broadly accepted that we need some sort of standard scheduler.

As part of [P3109R0], `system_context` was voted as a must-have for the initial release of senders/receivers. It provides a convenient and scalable way of spawning concurrent work for the users of senders/receivers.

As noted in [P2079R1], an earlier revision of this paper, the `static_thread_pool` included in later revisions of [P0443R14] had many shortcomings. This was removed from [P2300R10] based on that and other input.

One of the biggest problems with local thread pools is that they lead to CPU oversubscription. This introduces a performance problem for complex systems that are composed from many independent parts.

Another problem that system context is aiming to solve is the composability of components that may rely on different parallel engines. An application might have multiple parts, possibly in different binaries; different parts of the application may not know of each other. Thus, different parts of the application might use different parallel engines. This can create several problems:

- oversubscription because of different thread pools
- problems with nested parallel loops (one parallel loop is called from the other)
- problems related to interaction between different parallel engines
- etc.

To solve these problems we propose a parallel execution context that:

- can be shared between multiple parts of the application
- does not suffer from oversubscription
- can integrate with the OS scheduler
- (potentially) can be replaced by the user to compose well with other parallel runtimes

§ 2.1. Design overview

The system context is a parallel execution context of undefined size, supporting explicitly *parallel forward progress*.

The execution resources of the system context are envisioned to be shared across all binaries in the same process. System scheduler works best with CPU-intensive workloads, and thus, limiting oversubscription is a key goal.

By default, the system context should be able to use the OS scheduler, if the OS has one. On systems where the OS scheduler is not available, the system context will have a generic implementation that acts like a thread pool.

For enabling the users to hand-tune the performance of their applications, and for fulfilling the composability requirements, the system context should be replaceable. The user should be able to replace the default implementation of the system context with a custom one that fits their needs. The replaceability mechanism are left to be implementation-defined.

Other key concerns of this design are:

- Extensibility: being able to extend the design to work with new additions to the senders/receivers framework.
- Lifetime: as `system_context` is a global resource, we need to pay attention to the lifetime of this resource.
- Performance: as we envision this to be used in many cases to spawn concurrent work, performance considerations are important.

§ 3. Examples

As a simple parallel scheduler we can use it locally, and `sync_wait` on the work to make sure that it is complete. With forward progress delegation this would also allow the scheduler to delegate work to the blocked thread. This example is derived from the Hello World example in [\[P2300R10\]](#). Note that it only adds a well-defined context object, and queries that for the scheduler. Everything else is unchanged about the example.

```
using namespace = std::execution;

scheduler auto sch = get_system_scheduler();

sender auto begin = schedule(sch);
sender auto hi = then(begin, []{
    std::cout << "Hello world! Have an int.";
    return 13;
});
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });

auto [i] = std::this_thread::sync_wait(add_42).value();
```

We can structure the same thing using `on`, which better matches structured concurrency:

```
using namespace std::execution;

scheduler auto sch = get_system_scheduler();

sender auto hi = then(just(), []{
    std::cout << "Hello world! Have an int.";
    return 13;
});
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });

auto [i] = std::this_thread::sync_wait(on(sch, add_42)).value();
```

The `system_scheduler` customizes `bulk`, so we can use `bulk` dependent on the scheduler. Here we use it in structured form using the parameterless `get_scheduler` that retrieves the scheduler from the receiver, combined with `on`:

```
using namespace std::execution;

auto bar() {
    return
        let_value(
            read_env(get_scheduler()),      // Fetch scheduler from receiver.
            [](auto current_sched) {
                return bulk(
                    current_sched.schedule(),
                    1,                      // Only 1 bulk task as a lazy way of ma
                    [](auto idx){
                        std::cout << "Index: " << idx << "\n";
                    })
            });
}

void foo()
{
    auto [i] = std::this_thread::sync_wait(
        on(
            get_system_scheduler(),        // Start bar on the system_scheduler
            bar()))                        // and propagate it through the recei
        .value();
}
```

Use `async_scope` and a custom system context implementation linked in to the process (through a mechanism undefined in the example). This might be how a given platform exposes a custom context. In this case we assume it has no threads of its own and has to take over the main thread through an custom `drive()` operation that can be looped until a callback requests `exit` on the context.

```
using namespace std::execution;

int result = 0;

{
    async_scope scope;
    scheduler auto sch = get_system_scheduler();
```

```

sender auto work =
    then(just(), [&](auto sched) {

        int val = 13;

        auto print_sender = then(just(), [val]{
            std::cout << "Hello world! Have an int with value: " << val << "\n"
        });

        // spawn the print sender on sched to make sure it
        // completes before shutdown
        scope.spawn(on(sch, std::move(print_sender)));

        return val;
    });

scope.spawn(on(sch, std::move(work)));

// This is custom code for a single-threaded context that we have replaced
// We need to drive it in main.
// It is not directly sender-aware, like any pre-existing work loop, but
// does provide an exit operation. We may call this from a callback chain
// after the scope becomes empty.
// We use a temporary terminal_scope here to separate the shut down
// operation and block for it at the end of main, knowing it will complete
async_scope terminal_scope;
terminal_scope.spawn(
    scope.on_empty() | then([](my_os::exit(sch))));
my_os::drive(sch);
std::this_thread::sync_wait(terminal_scope);
};

// The scope ensured that all work is safely joined, so result contains 13
std::cout << "Result: " << result << "\n";

// and destruction of the context is now safe

```

§ 4. Design

```
system_scheduler get_system_scheduler();

class system_scheduler { // exposition only
public:
    system_scheduler() = delete;
    ~system_scheduler();

    system_scheduler(const system_scheduler&);
    system_scheduler(system_scheduler&&);
    system_scheduler& operator=(const system_scheduler&);
    system_scheduler& operator=(system_scheduler&&);

    bool operator==(const system_scheduler&) const noexcept;

    forward_progress_guarantee query(get_forward_progress_guarantee_t) const;

    impl-defined-system_sender schedule() const noexcept;
    // customization for bulk
};

class impl-defined-system_sender { // exposition only
public:
    system_scheduler query(get_completion_scheduler_t<set_value_t>) const noexcept;
    system_scheduler query(get_completion_scheduler_t<set_stopped_t>) const;

    template<receiver R>
    requires receiver_of<R>
    impl-defined-operation_state connect(R&&) && noexcept(std::is_nothrow_constructible<R, R&&>);
};
```

- `get_system_scheduler()` returns a scheduler that provides a view on some underlying execution context supporting *parallel forward progress*, with at least one thread of execution (which may be the main thread).
- two objects returned by `get_system_scheduler()` may share the same execution context. If work submitted by one can consume the underlying thread pool, that can block progress of another.
- if `Sch` is the type of object returned by `get_system_scheduler()`, then:
 - `Sch` is implementation-defined, but must be nameable.

- `Sch` models the `scheduler` concept.
- `Sch` implements the `get_forward_progress_guarantee` query to return `parallel`.
- `Sch` implements `schedule` customization point to return an implementation-defined `sender` type.
- `schedule` calls on `Sch` are non-blocking operations.
- `Sch` implements the `bulk` CPO to customize the `bulk` sender adapter such that:
 - when `execution::set_value(r, args...)` is called on the created `receiver`, an agent is created with parallel forward progress on the underlying system context for each `i` of type `Shape` from `0` to `sh`, where `sh` is the shape parameter to the `bulk` call, that calls `f(i, args...)`.
- if `sch` is an object returned by `get_system_scheduler()`, then:
 - the lifetime of `sch` does not have to outlive work submitted to it.
 - `sch` is both move and copy constructible and assignable.
 - if `sch2` is another object returned by `get_system_scheduler()`, then `sch == sch2` is defined and always evaluates to `true`.
 - if the underlying system context is unable to make progress on work created through `sch`, and the sender retrieved from `sch` is connected to a `receiver` that supports the `get_delegatee_scheduler` query, work may be scheduled on the `scheduler` returned by `get_delegatee_scheduler` at the time of the call to `start`, or at any later point before the work completes.
- if `snd` is a sender obtained from the `scheduler` returned by `get_system_scheduler()`, and `Snd` its type, then:
 - `Snd` is implementation-defined, but must be nameable.
 - `Snd` models the `sender` concept.
 - `Snd` implements the `get_completion_scheduler` query for the value and done channel where it returns a type that is logically a pair of an object that compares equal to itself, and a representation of delegatee scheduler that may be obtained from receivers connected with the sender.
 - connecting `snd` to a `receiver` object and calling `start()` on the resulting operation state are non-blocking operations.
 - if `snd` is connected with a `receiver` that supports the `get_stop_token` query and if that `stop_token` is stopped, operations on which `start` has been called, but are not yet run-

ning (and are hence not yet guaranteed to make progress) **must** complete with `set_stopped` as soon as is practical.

§ 5. Design discussion and decisions

§ 5.1. To drive or not to drive

On single-threaded systems (e.g., freestanding implementations) or on systems in which the main thread has special significance (e.g., to run the Qt main loop), it's important to allow scheduling work on the main thread. For this, we need the main thread to *drive* work execution.

The earlier version of this paper, [P2079R2], included `execute_all` and `execute_chunk` operations to integrate with senders. In this version we have removed them because they imply certain requirements of forward progress delegation on the system context and it is not clear whether or not they should be called. We envision a separate paper that adds the support for drive-ability, which is decoupled by this paper.

We can simplify this discussion to a single function:

```
void drive(system_context& ctx, sender auto snd);
```

Let's assume we have a single-threaded environment, and a means of customizing the `system_context` for this environment. We know we need a way to donate `main`'s thread to this context, it is the only thread we have available. Assuming that we want a `drive` operation in some form, our choices are to:

- define our `drive` operation, so that it is standard, and we use it on this system.
- or allow the customization to define a custom `drive` operation related to the specific single-threaded environment.

With a standard `drive` of this sort (or of the more complex design in [P2079R2]) we might write an example to use it directly:

```
system_context ctx;  
auto snd = on(ctx, doWork());  
drive(ctx, std::move(snd));
```

Without drive, we rely on an `async_scope` to spawn the work and some system-specific drive operation:

```
system_context ctx;
async_scope scope;
auto snd = on(ctx, doWork());
scope.spawn(std::move(snd));
custom_drive_operation(ctx);
```

Neither of the two variants is very portable. The first variant requires applications that don't care about drive-ability to call `drive`, while the second variant requires custom plumbing to tie the main thread with the system scheduler.

We envision a new paper that adds support for a *main scheduler* similar to the *system scheduler*. The main scheduler, for hosted implementations would be typically different than the system scheduler. On the other hand, on freestanding implementations, the main scheduler and system scheduler can share the same underlying implementation, and both of them can execute work on the main thread; in this mode, the main scheduler is required to be driven, so that system scheduler can execute work.

Keeping those two topic as separate papers allows to make progress independently.

§ 5.2. Freestanding implementations

This paper paid attention to freestanding implementations, but doesn't make any wording proposals for them. We express a strong desire for the system scheduler to work on freestanding implementations, but leave the details to a different paper.

We envision that, a followup specification will ensure that the system scheduler will work in freestanding implementations by sharing the implementation with the main scheduler, which is driven by the main thread.

§ 5.3. Making system context implementation-defined and replaceable

The system context aims to allow people to implement an application that is dependent only on parallel forward progress and to port it to a wide range of systems. As long as an application does not rely on concurrency, and restricts itself to only the system context, we should be able to scale from single threaded systems to highly parallel systems.

In the extreme, this might mean porting to an embedded system with a very specific idea of an execution context. Such a system might not have a multi-threading support at all, and thus the system context not only runs with single thread, but actually runs on the system's only thread. We might build the context on top of a UI thread, or we might want to swap out the system-provided implementation with one from a vendor (like Intel) with experience writing optimized threading runtimes.

The latter is also important for the composability of the existing code with the `system_context`, i.e., if Intel Threading building blocks (oneTBB) is used by somebody and they want to start using `system_context` as well, it's likely that the users want to replace `system_context` implementation with oneTBB because in that case they would have one thread pool and work scheduler underneath.

We should allow customization of the system context to cover this full range of cases. For a whole platform this is relatively simple. We assume that everything is an implementation-defined type. The `system_context` itself is a named type, but in practice is implementation-defined, in the same way that `std::vector` is implementation-defined at the platform level.

Other situations may offer a little less control. If we wish Intel to be able to replace the system thread pool with TBB, or Adobe to customize the runtime that they use for all of Photoshop to adapt to their needs, we need a different customization mechanism.

To achieve this we see options:

1. Link-time replaceability. This could be achieved using weak symbols, or by choosing a runtime library to pull in using build options.
2. Run-time replaceability. This could be achieved by subclassing and requiring certain calls to be made early in the process.
3. Compile-time replaceability. This could be achieved by importing different headers, by macro definitions on the command line or various other mechanisms.

Link-time replaceability has the following characteristics:

- Pro: we have precedence in the standard: this is similar to replacing `operator new`.
- Pro: more predictable, in that it can be guaranteed to be application-global.
- Pro: some of the type erasure and indirection can be removed in practice with link-time optimization.
- Con: it requires defining the ABI and thus, in some cases, would require some type erasure and some inefficiency.

- Con: harder to get it correctly with shared libraries (e.g., DLLs might have different replaced versions of the system scheduler).
- Con: the replacement might depend on the order of linking.

Run-time replaceability has the following characteristics:

- Pro: we have precedence in the standard: this is similar to `std::set_terminate()`.
- Pro: easier to achieve consistent behavior on applications with shared libraries (e.g., Windows has the same version of C++ standard library in DLL).
- Pro: a program can have multiple implementations of system scheduler.
- Con: race conditions between replacing the system scheduler and using it to spawn work.
- Con: implies going over an ABI, and cannot be optimized at link-time.
- Con: different implementation may allocate resources for the system scheduler at startup, and then, at the start of main, the implementation is replaced (this is mainly a QOI issue).
- Con: requires strict lifetime and ownership control to be safe, and for the user to do the right thing explicitly.

Compile-time replaceability has the following characteristics:

- Pro: users can do this with a type-def that can be used everywhere and switched.
- Con: potential problems with ODR violations.
- Con: doesn't support shareability across different binaries of the same process

The paper considers compile-time replaceability as not being a viable option because it easily breaks one of the fundamental design principles of a `system_context`, i.e. having one, shared, application-wide execution context, which avoids oversubscription.

Replaceability is also part of the [\[P2900R8\]](#) proposal for the contract-violation handler. The paper proposes that whether the handler is replaceable to be implementation-defined. If an implementation chooses to support replaceability, it shall be done similar to replacing the global `operator new` and `operator delete` (link-time replaceability).

The feedback we received from Microsoft, is that they will likely not support replaceability on their platforms. They would prefer that we offer implementations an option to not implement replaceability. Moreover, for systems where replaceability is supported they would prefer to make the replaceability mechanism to be implementation defined.

The authors disagree with the idea that replaceability is not needed for Windows platforms (or other platforms that provide an OS scheduler). The OS scheduler is optimized for certain workloads, and it's not the best choice for all workloads. This way not providing replaceability options have the following drawbacks:

- it limits the ability to hand-tune the performance of the application (when system scheduler is used);
- it limits the `system_scheduler` ability to be used in for CPU-intensive workloads;
- it limits the `system_scheduler` ability to be used for platforms with accelerators;
- it limits the composability of the system context with other parallel runtimes (while avoiding oversubscription).

For this reasons, the authors encourage all platforms to support replaceability of the system context.

However, in accordance with the feedback, the paper proposes the following:

- whether the system context is replaceable or not is implementation-defined.
- the replaceability mechanism (if the implementation decides to support it), including the interfaces that a backend should implement is implementation-defined.

During the development of this paper, we received constant feedback that the replaceability mechanism should be standardized, even if we standardize just the interfaces that a backend needs to implement (leaving the replaceability mechanism to be implementation-defined). However, as time went by, more and more people think that agreeing on the same replaceability API shape is going to be problematic. Here are a few reasons why:

- Different standard library vendors might have different needs; if the replaceability API is too generic to cover all the needs, we compromise on performance. Example:
 - for a simple `schedule` operation, some implementations would want cancellation in the backend, some would not (cancellation is better to be handled in the frontend).
 - including cancellation in the replaceability API would satisfy the needs for those who want cancellation but that would add extra performance penalties
 - in general, including a runtime environment in the backend may be costly, and some implementations may not need it
- Even if we specify the replaceability API, somebody who implements a backend for system context may still need to interact with implementation-defined abstractions, as the overall replaceability mechanism is implementation-defined.

- As certain vendors won't implement replaceability, defining a standard API for replaceability has diminishing returns.

§ 5.4. Extensibility

The `std::execution` framework is expected to grow over time. We expect to add time-based scheduling, async I/O, priority-based scheduling, and other for now unforeseen functionality. The `system_context` framework needs to be designed in such a way that it allows for extensibility.

Whatever the replaceability mechanism is, we need to ensure that new features can be added to the system context in a backwards-compatible manner.

There are two levels in which we can extend the system context:

1. Add more types of schedulers, beside the system scheduler.
2. Add more features to the existing scheduler.

The first type of extensibility can easily be solved by adding new getters for the new types of schedulers. Different types of schedulers should be able to be replaced separately; e.g., one should be able to replace the I/O scheduler without replacing the system scheduler. The discussed replaceability mechanisms support this.

The second type of extensibility can also be easily achieved, but, at this point, it's beside of the scope of this paper. Next section provides more details.

§ 5.5. API for implementing custom system contexts

A proper implementation of the system scheduler that meets all the goals expressed in the paper needs to be divided into two parts: "host" and "backend". The host part implements the API defined in this paper and calls the backend for the actual implementation. The backend provides the actual implementation of the system context (e.g., use Grand Central Dispatch or Windows Thread Pool).

As we need to switch between different backend, we need a "stable" type-erased API between these two parts.

An example of such an API can be found in [stdexec](#) repository.

§ 5.6. Shareability

One of the motivations of this paper is to stop the proliferation of local thread pools, which can lead to CPU oversubscription. If multiple binaries are used in the same process, we don't want each binary to have its own implementation of system context. Instead, we would want to share the same underlying implementation.

The recommendation of this paper is to leave the details of shareability to be implementation-defined or unspecified.

§ 5.7. Performance

To support shareability and replaceability, system context calls may need to go across binary boundaries, over the defined API. A common approach for this is to have COM-like objects. However, the problem with that approach is that it requires memory allocation, which might be a costly operation. This becomes problematic if we aim to encourage programmers to use the system context for spawning work in a concurrent system.

While there are some costs associated with implementing all the goals stated here, we want the implementation of the system context to be as efficient as possible. For example, a good implementation should avoid memory allocation for the common case in which the default implementation is utilized for a platform.

This paper cannot recommend the specific implementation techniques that should be used to maximize performance; these are considered Quality of Implementation (QOI) details.

Standardizing a replaceability API generated many discussions related to performance. In the end, we reached the conclusion that, if we want to implement all the features that backends may need, we may need to sacrifice performance. This is one of the chief reasons for which this paper doesn't standardize the replaceability API.

§ 5.8. Lifetime

Underneath the system scheduler, there is a singleton of some sort. We need to specify the lifetime of this object and everything that derives from it.

Revision R4 of the paper mandates that the lifetime of any `system_context` must be fully contained within the lifetime of `main()`. The reasoning behind this is that ensuring proper construction and destruction order of static objects is typically difficult in practice. This is especially challenging during

the destruction of static objects; and, by symmetry, we also did not want to guarantee the lifetime of the system scheduler before `main()`. We argued that if we took a stricter approach, we could always relax it later.

We received feedback that this was too strict. First, there are many applications where the C++ part does not have a `main()` function. Secondly, this can be considered a quality of implementation issue; implementations can always use a Phoenix singleton pattern to ensure that the underlying system context object remains alive for the duration of the entire program.

R5 revision of the paper relaxes the lifetime requirements of the system scheduler. The system scheduler can now be used in any part of a C++ program.

§ 5.9. Need for the `system_context` class

Our goal is to expose a global shared context to avoid oversubscription of threads in the system and to efficiently share a system thread pool. Underneath the `system_context` there is a singleton of some sort, potentially owned by the OS.

The question is how we expose the singleton. We have a few obvious options:

- Explicit context objects, as we've described in R2, R3 and R4 of this paper, where a `system_context` is constructed as any other context might be, and refers to a singleton underneath.
- A global `get_system_context()` function that obtains a `system_context` object, or a reference to one, representing the singleton explicitly.
- A global `get_system_scheduler()` function that obtains a scheduler from some singleton system context, but does not explicitly expose the context.

In R4 and earlier revisions, we opted for an explicit context object. The reasoning was that providing explicit contexts makes it easier to understand the lifetime of the schedulers. However, adding this extra class does not affect how one would reason about the lifetime of the schedulers or the work scheduled on them. Therefore, introducing an artificial scope object becomes an unnecessary burden.

There were also arguments made for adding `system_context` so that we can later add properties to it, that don't necessarily belong to `system_scheduler`. However, if we would later find such properties, nothing prevents us to add `system_context` class later, and make `get_system_scheduler()` return `get_system_context().get_scheduler()`.

Thus, the paper simply proposes a `get_system_scheduler()` function that returns a the system scheduler. The system context is implementation-defined and not exposed to the user.

§ 5.10. Priorities

It's broadly accepted that we need some form of priorities to tweak the behavior of the system context. This paper does not include priorities, though early drafts of R2 did. We had different designs in flight for how to achieve priorities and decided they could be added later in either approach.

The first approach is to expand one or more of the APIs. The obvious way to do this would be to add a priority-taking version of `system_context::get_scheduler()`:

```
implementation-defined-system_scheduler get_scheduler();  
implementation-defined-system_scheduler get_scheduler(priority_t priority);
```

This approach would offer priorities at scheduler granularity and apply to large sections of a program at once.

The other approach, which matches the receiver query approach taken elsewhere in [P2300R10] is to add a `get_priority()` query on the receiver, which, if available, passes a priority to the scheduler in the same way that we pass an `allocator` or a `stop_token`. This would work at task granularity, for each `schedule()` call that we connect a receiver to we might pass a different priority.

In either case we can add the priority in a separate paper. It is thus not urgent that we answer this question, but we include the discussion point to explain why they were removed from the paper.

§ 5.11. Reference implementation

The authors prepared a reference implementation in `stdexec`

A few key points of the implementation:

- The implementation is divided into two parts: "host" and "backend". The host part implements the API defined in this paper and calls the backend for the actual implementation. The backend provides the actual implementation of the system context.
- Allows link-time replaceability for `system_scheduler`. Provides examples on doing this.
- Allows run-time replaceability for `system_scheduler`. Provides examples on doing this.
- Defines a replaceability API between the host and backend parts. This way, one can easily extend this interface when new features need to be added to `system_context`.
- Uses preallocated storage on the host side, so that the default implementation doesn't need to allocate memory on the heap when adding new work to `system_scheduler`.

- Guarantees a lifetime of at least the duration of `main()`.
- As the default implementation is created outside of the host part, it can be shared between multiple binaries in the same process.
- uses a `static_thread_pool`-based implementation as a default on generic platforms (we have a patch that uses `libdispatch` as default implementation on MacOS; as the time of writing this paper revision, the patch is not yet merged on the mainline).

§ 5.12. Addressing received feedback

§ 5.12.1. Allow for system context to borrow threads

Early feedback on the paper from Sean Parent suggested a need for the system context to support a configuration where it carries no threads of its own and takes over the main thread. While in [P2079R2] we proposed `execute_chunk` and `execute_all`, these enforce a particular implementation on the underlying execution context. Instead, we simplify the proposal by removing this functionality and assuming that it is implemented by link-time or run-time replacement of the context. We assume that the underlying mechanism to drive the context, should one be necessary, is implementation-defined. This allows for custom hooks into an OS thread pool, or a simple `drive()` method in main.

As we discussed previously, a separate paper is supposed to take care of the drive-ability aspect.

§ 5.12.2. Allow implementations to use Grand Central Dispatch and Windows Thread Pool

In the current form of the paper, we allow implementations to define the best choice for implementing the system context for a particular system. This includes using Grand Central Dispatch on Apple platforms and Windows Thread Pool on Windows.

In addition, we propose implementations to allow the replaceability of the system context implementation. This means that users should be allowed to write their own system context implementations that depend on OS facilities or a necessity to use some vendor (like Intel) specific solutions for parallelism.

§ 5.12.3. Priorities and elastic pools

Feedback from Sean Parent:

There is so much in that proposal that is not specified. What requirements are placed on the system scheduler? Most system schedulers support priorities and are elastic (i.e., blocking in the system thread pool will spin up additional threads to some limit).

The lack of details in the specification is intentional, allowing implementers to make the best compromises for each platform. As different platforms have different needs, constraints, and optimization goals, the authors believe that it is in the best interest of the users to leave some of these details as Quality of Implementation (QOI) details.

§ 5.12.4. Implementation-defined may make things less portable

Some feedback gathered during discussions on this paper suggested that having many aspects of the paper to be implementation-defined would reduce the portability of the system context.

While it is true that people that would want to replace the system scheduler will have a harder time doing so, this will not affect the users of the system scheduler. They would still be able to use system context and system scheduler without knowing the implementation details of those.

We have a precedence in the C++ standard for this approach with the global allocator.

§ 6. Annex: A possible API for implementing custom system contexts

During the discussions on R4 and R5 of the paper, we have spent effort on a possible API for implementing custom system contexts. During the final phases of R5, we agreed that we should not standardize such an API. However, for reference, we include here the API discussed.

Implementations may support replaceability of system scheduler; it's implementation-defined whether the system scheduler is replaceable or not. If an implementation supports replaceability, the following API might be used. The way that the following interfaces are used to replace the system scheduler is implementation-defined. If the implementation of the system context does not support replaceability, the following API is not needed.

```
namespace std::system_context_replaceability {
    template <typename Interface>
    Interface* query_system_context();

    struct receiver {
        virtual ~receiver() = default;
    };
}
```

```

    receiver(const receiver&) = delete;
    receiver(receiver&&) = delete;
    receiver& operator=(const receiver&) = delete;
    receiver& operator=(receiver&&) = delete;

    virtual void set_value() noexcept = 0;
    virtual void set_error(std::exception_ptr) noexcept = 0;
    virtual void set_stopped() noexcept = 0;
};

struct bulk_item_receiver : receiver {
    virtual void start(uint32_t) noexcept = 0;
};

struct storage {
    void* data;
    uint32_t size;
};

struct system_scheduler {
    virtual ~system_scheduler() = default;

    virtual void schedule(receiver*, storage, inplace_stop_token) noexcept
    virtual void bulk_schedule(uint32_t, bulk_item_receiver*, storage, inp
};
}

```

- **NOTE:** for the current exposition we call *backend* the part of the application that implements a custom system context, and *host* the part of the application that uses the custom system context. The *host* part consists of library code that implements the user-facing API to call the interfaces implemented by the backend.
- `query_system_context` is a function that returns a pointer to an object that implements the `Interface` concept.
 - `query_system_context` must have a valid instantiation for the `system_scheduler` interface; it returns the underlying system scheduler implementation that is shared by all user-facing `system_context` objects.

- is not required for `query_system_context` to be instantiable for interfaces like `receiver`, `bulk_item_receiver`, or `storage`.
- **NOTE:** in the future, this mechanism may be used to query different types of system context objects, like I/O schedulers, priority schedulers, time schedulers, main scheduler, etc. It can also be used to query implementation-specific interfaces.
- **NOTE:** `receiver`, `bulk_item_receiver`, and `storage` are interfaces that are implemented by the host side; only `system_scheduler` is expected to be implemented by a system context implementation.
- the names of the fields of `storage` class are implementation-defined.
- if `sch` is an object that implements `system_scheduler` interface (and can be returned by `query_system_context`), then the following must be true:
 - the users of `sch` must not destroy the object.
 - if `sch.schedule()` is called passing `r` of type `receiver*`, `st` of type `storage` and `pf` of type `inplace_stop_token`, then:
 - at least one of `set_value`, `set_error`, or `set_stopped` must be eventually called on `r`;
 - `set_value` is called to signal a successful scheduling of the work; it must be called on a thread belonging to system execution context;
 - if `set_value` cannot be called, then `set_error` must be called to signal the scheduling error;
 - `set_stopped` may be called on `r` to signal that the execution of work is no longer needed;
 - the `storage` object `st` represents a memory region that starts at `st.data` and has `st.size` bytes.
 - the storage represented by `st` must be valid until the `receiver` object `r` is signaled.
 - the implementation may use the storage represented by `st` to store data needed for the scheduling operation.
 - **NOTE:** if `s.stop_requested()` returns `true`, `set_stopped` may be called; but this is not guaranteed.
 - if `sch.bulk_schedule()` is called passing `n` of type `uint32_t`, `r` of type `bulk_item_receiver*`, `st` of type `storage` and `t` of type `inplace_stop_token`, then:

- at least one of `set_value`, `set_error`, or `set_stopped` must be eventually called on `r`;
 - `set_value` is called to signal a successful scheduling of the work; it must be called on a thread belonging to system execution context;
 - if `set_value` cannot be called, then `set_error` must be called to signal the scheduling error;
 - `set_stopped` may be called on `r` to signal that the execution of work is no longer needed;
- the `storage` object `st` represents a memory region that starts at `st.data` and has `st.size` bytes.
 - the storage represented by `st` must be valid until the `receiver` object `r` is signaled.
 - the implementation may use the storage represented by `st` to store data needed for the scheduling operation.
- **NOTE:** if `s.stop_requested()` returns `true`, `set_stopped` may be called; but this is not guaranteed.
- if `set_value` is called on `r`, then `start` must be called on `r` `n` times, where `n` is the value passed to `bulk_schedule`.
 - the `start` method is called on `r` to signal the start of the work for the `i`-th item in the bulk operation, where `i` is in the range `[0, n)`.
 - the `start` method must be called on a thread belonging to the system execution context.
 - the `start` method must be called before any methods on `r` is called.
- if in the process of calling `start` on `r`, the implementation detects that the work cannot be started, then `set_error` must be called on `r` to signal the error; in this case `r` may not get all the expected `n` calls to `start`.
- if in the process of calling `start` on `r`, the scheduling process is cancelled, then `set_stopped` must be called on `r` to signal the cancellation; in this case `r` may not get all the expected `n` calls to `start`.

§ References

§ Informative References

[P0443R14]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysisen, H. Carter Edwards, Gordon Brown, D. S. Hollman. *A Unified Executors Proposal for C++*. 15 September 2020. URL: <https://wg21.link/p0443r14>

[P2079R1]

Ruslan Arutyunyan, Michael Voss. *Parallel Executor*. 15 August 2020. URL: <https://wg21.link/p2079r1>

[P2079R2]

Lee Howes, Ruslan Arutyunyan, Michael Voss. *System execution context*. 15 January 2022. URL: <https://wg21.link/p2079r2>

[P2300R10]

Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. *std::execution*. 28 June 2024. URL: <https://wg21.link/p2300r10>

[P2900R8]

Joshua Berne, Timur Doumler, Andrzej Krzemiński. *Contracts for C++*. 27 July 2024. URL: <https://wg21.link/p2900r8>

[P3109R0]

Lewis Baker, Eric Niebler, Kirk Shoop, Lucian Radu. *A plan for std::execution for C++26*. 12 February 2024. URL: <https://wg21.link/p3109r0>